

Lab 3: Network Traffic Prediction Using Supervised Learning

Course: ITAI Telecommunications/Networking Course

Student: Brandie Griffin

Professor: Dr. Tawanda Chiyangwa

Date: June 2026

Introduction

Network operators must predict future traffic demand to ensure that communication networks remain reliable and efficient. Accurate traffic prediction helps telecommunications companies allocate resources, prevent congestion, and improve the overall quality of service provided to customers.

In this lab, a supervised machine learning approach is used to predict network traffic levels based on historical traffic patterns. A synthetic telecommunications dataset containing hourly traffic measurements is analyzed. Features such as hour of the day, day of the week, business hours, previous traffic values, and rolling averages are used to train a Random Forest Regressor model. The model is then evaluated using Mean Squared Error (MSE) and R^2 metrics to determine its prediction accuracy.

The objective of this lab is to demonstrate how machine learning techniques can be applied to network traffic forecasting and network resource management.

Step 1: Import Required Libraries

The following libraries are used for data generation, visualization, machine learning, and model evaluation.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
import seaborn as sns
```

✓ Step 2: Generate Synthetic Network Traffic Dataset

This section creates a synthetic hourly telecommunications traffic dataset containing daily, weekly, and annual traffic patterns.

```
# Generate synthetic telecom traffic data
np.random.seed(42)
dates = pd.date_range(start='2023-01-01', end='2023-12-31', freq='h')
n_samples = len(dates)

# Create realistic traffic patterns
hour = dates.hour
day_of_week = dates.dayofweek
day_of_year = dates.dayofyear

# Base traffic with daily and weekly patterns
base_traffic = (
    50 +
    20 * np.sin(2 * np.pi * hour / 24) +
    10 * np.sin(2 * np.pi * day_of_week / 7) +
    5 * np.sin(2 * np.pi * day_of_year / 365) +
    np.random.normal(0, 5, n_samples)
)

business_hours = (hour >= 9) & (hour <= 17) & (day_of_week < 5)
base_traffic += business_hours * 15
```

```
# Create DataFrame
traffic_data = pd.DataFrame({
    'timestamp': dates,
    'hour': hour,
    'day_of_week': day_of_week,
    'is_weekend': day_of_week >= 5,
    'is_business_hours': business_hours,
    'traffic_gbps': np.maximum(base_traffic, 0)
})

# Feature engineering
traffic_data['traffic_lag_1h'] = traffic_data['traffic_gbps'].shift(1)
traffic_data['traffic_lag_24h'] = traffic_data['traffic_gbps'].shift(24)

traffic_data['traffic_rolling_7d'] = traffic_data['traffic_gbps'].rolling(
    window=24*7,
    center=True
).mean()

# Remove rows with NaN values
traffic_data = traffic_data.dropna()
```

```
print("Dataset shape:", traffic_data.shape)
print("\nFirst few rows:")
print(traffic_data.head())
```

Dataset shape: (8570, 9)

First few rows:

	timestamp	hour	day_of_week	is_weekend	is_business_hours	\
84	2023-01-04 12:00:00	12	2	False	True	
85	2023-01-04 13:00:00	13	2	False	True	
86	2023-01-04 14:00:00	14	2	False	True	
87	2023-01-04 15:00:00	15	2	False	True	
88	2023-01-04 16:00:00	16	2	False	True	

	traffic_gbps	traffic_lag_1h	traffic_lag_24h	traffic_rolling_7d
84	71.050823	77.678321	70.680542	54.166101
85	67.408125	71.050823	66.971737	54.147588
86	69.670302	67.408125	57.544738	54.132844
87	62.594911	69.670302	52.953244	54.090667
88	55.123982	62.594911	59.818534	54.024632

✓ Step 3: Visualize Network Traffic Patterns

The traffic dataset is visualized to better understand daily and weekly traffic patterns.

```
plt.figure(figsize=(15, 8))

plt.subplot(2, 2, 1)
plt.plot(traffic_data['timestamp'][:24*7], traffic_data['traffic_gbps'][:24*7])
plt.title('Traffic Pattern - One Week')
plt.xlabel('Time')
plt.ylabel('Traffic (Gbps)')
plt.xticks(rotation=45)

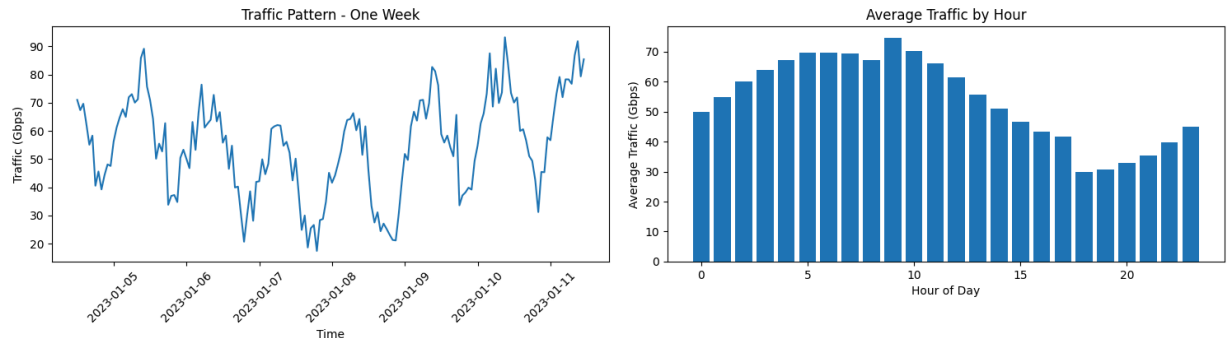
plt.subplot(2, 2, 2)
hourly_avg = traffic_data.groupby('hour')['traffic_gbps'].mean()
plt.bar(hourly_avg.index, hourly_avg.values)
plt.title('Average Traffic by Hour')
plt.xlabel('Hour of Day')
plt.ylabel('Average Traffic (Gbps)')

plt.subplot(2, 2, 3)
daily_avg = traffic_data.groupby('day_of_week')['traffic_gbps'].mean()
days = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']
plt.bar(days, daily_avg.values)
plt.title('Average Traffic by Day of Week')
plt.ylabel('Average Traffic (Gbps)')

plt.subplot(2, 2, 4)
```

```
plt.scatter(
    traffic_data['traffic_lag_24h'],
    traffic_data['traffic_gbps'],
    alpha=0.5
)
plt.title('Traffic Autocorrelation (24h lag)')
plt.xlabel('Traffic 24h ago (Gbps)')
plt.ylabel('Current Traffic (Gbps)')

plt.tight_layout()
plt.show()
```



✓ Step 4: Feature Selection and Data Preparation

Relevant features and the target variable are selected for model training.

```
feature_columns = [
    'hour',
    'day_of_week',
    'is_weekend',
    'is_business_hours',
    'traffic_lag_1h',
    'traffic_lag_24h',
    'traffic_rolling_7d'
]

X = traffic_data[feature_columns]
y = traffic_data['traffic_gbps']

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    shuffle=False
)
```

Step 5: Train the Random Forest Regression Model

A Random Forest Regressor is trained using historical traffic information to predict future network traffic levels.

```
rf_model = RandomForestRegressor(  
    n_estimators=100,  
    max_depth=10,  
    random_state=42,  
    n_jobs=-1  
)  
  
rf_model.fit(X_train, y_train)
```

RandomForestRegressor ⓘ ?
RandomForestRegressor(max_depth=10, n_jobs=-1, random_state=42)

Step 6: Model Prediction and Performance Evaluation

The trained model is evaluated using Mean Squared Error (MSE) and R^2 metrics to measure prediction accuracy.

```
y_pred_train = rf_model.predict(X_train)  
y_pred_test = rf_model.predict(X_test)  
  
train_mse = mean_squared_error(y_train, y_pred_train)  
test_mse = mean_squared_error(y_test, y_pred_test)  
  
train_r2 = r2_score(y_train, y_pred_train)  
test_r2 = r2_score(y_test, y_pred_test)  
  
print(f"\nModel Performance:")  
print(f"Training MSE: {train_mse:.2f}")  
print(f"Testing MSE: {test_mse:.2f}")  
print(f"Training R²: {train_r2:.3f}")  
print(f"Testing R²: {test_r2:.3f}")
```

```
Model Performance:  
Training MSE: 17.59  
Testing MSE: 34.64  
Training R²: 0.946  
Testing R²: 0.893
```

Step 7: Feature Importance Analysis

Feature importance analysis identifies which variables have the greatest influence on network traffic predictions.

Step 7: Feature Importance Analysis

Feature importance analysis identifies which variables have the greatest influence on network traffic predictions.

```
feature_importance = pd.DataFrame({
    'feature': feature_columns,
    'importance': rf_model.feature_importances_
}).sort_values('importance', ascending=False)

print("\nFeature Importance:")
print(feature_importance)
```

```
Feature Importance:
      feature  importance
4  traffic_lag_1h    0.815706
0           hour    0.077702
5  traffic_lag_24h    0.035665
1    day_of_week    0.027819
6  traffic_rolling_7d    0.023439
3  is_business_hours    0.017457
2         is_weekend    0.002211
```

✓ Step 8: Compare Predicted and Actual Traffic

The model predictions are compared with actual network traffic values to visualize prediction performance.

```
plt.figure(figsize=(15, 6))

test_indices = X_test.index[:168]

plt.subplot(1, 2, 1)
plt.plot(
    test_indices,
    y_test.loc[test_indices],
    label='Actual',
    linewidth=2
)
```

```

plt.plot(
    test_indices,
    y_pred_test[:len(test_indices)],
    label='Predicted',
    linewidth=2,
    alpha=0.8
)

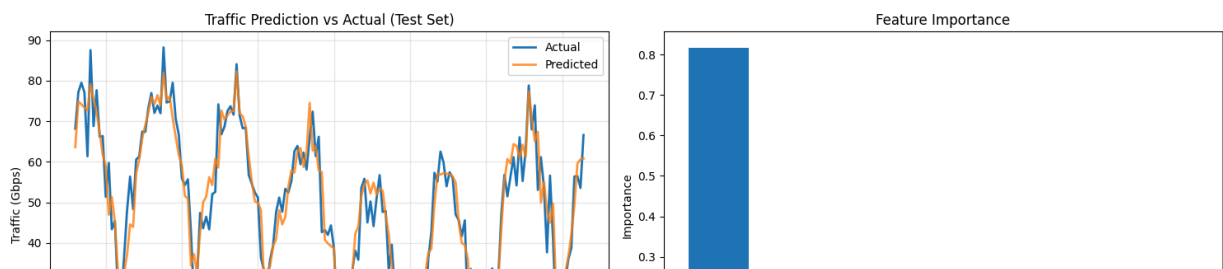
plt.title('Traffic Prediction vs Actual (Test Set)')
plt.xlabel('Time Index')
plt.ylabel('Traffic (Gbps)')
plt.legend()
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
plt.bar(
    feature_importance['feature'],
    feature_importance['importance']
)

plt.title('Feature Importance')
plt.xlabel('Features')
plt.ylabel('Importance')
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()

```



Results Discussion

The Random Forest model performed well in predicting network traffic levels. The model achieved a training R^2 score of 0.946 and a testing R^2 score of 0.893, indicating strong predictive accuracy and good generalization to unseen data. The relatively low Mean Squared Error values further suggest that prediction errors remained small throughout the evaluation process.

Feature importance analysis revealed that traffic from the previous hour was the most influential factor in predicting current traffic levels. Other important variables included the hour of the day, traffic from 24 hours earlier, and day of the week. These findings are

consistent with real telecommunications networks where traffic patterns tend to follow predictable daily and weekly cycles.

The prediction graph showed that the model's forecasts closely followed actual traffic trends, demonstrating the effectiveness of Random Forest regression for network traffic forecasting.

Conclusion

In this laboratory exercise, a Random Forest Regression model was developed to predict network traffic levels using historical telecommunications data. The dataset incorporated daily, weekly, and seasonal traffic patterns along with engineered features such as lag values and rolling averages.

The trained model achieved strong prediction accuracy, with a testing R^2 score of 0.893, demonstrating its ability to generalize to unseen traffic data. Feature importance analysis showed that recent traffic history was the most significant predictor of future traffic levels. The close alignment between predicted and actual traffic values further confirmed the effectiveness of the model.

Overall, this lab demonstrated how supervised machine learning techniques can be applied to network traffic forecasting. Such predictive models can assist network administrators in capacity planning, congestion management, and resource optimization within modern telecommunications networks.

References

Scikit-learn Developers. (2025). RandomForestRegressor. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Pandas Development Team. (2025). Pandas documentation. <https://pandas.pydata.org/docs/>

NumPy Developers. (2025). NumPy documentation. <https://numpy.org/doc/>